
What is this?

This repository provides some testing environment for NoSQL database systems, so that you can test how the query languages work.

Usage

Just use the scripts with `./run_<database>.sh` in this directory. The scripts will start a docker container with the database system, then it will create some test data, and then it will enter a shell where you can write some queries

When you are done you can kill the containers with `docker kill <database>; docker rm <database>`

Neo4J

The database is created with the following nodes and connections:

```
1 CREATE
2 (n1:Node {name:1}),
3 (n2:Node {name:2}),
4 (n3:Node {name:3}),
5 (n4:Node {name:4}),
6 (n5:Node {name:5}),
7 (n6:Node {name:6}),
8 (n7:Node {name:7}),
9 (n8:Node {name:8}),
10
11 (n1)-[:LINK]->(n2),
12 (n2)-[:LINK]->(n3),
13 (n3)-[:LINK]->(n1),
14 (n3)-[:LINK]->(n4),
15 (n3)-[:LINK]->(n5),
16 (n4)-[:LINK]->(n5),
17 (n4)-[:LINK]->(n7),
18 (n5)-[:LINK]->(n6),
19 (n6)-[:LINK]->(n4),
20 (n6)-[:LINK]->(n7),
21 (n6)-[:LINK]->(n8),
22 (n7)-[:LINK]->(n8);
```

Queries Returns nodes a, b, c matching this connection pattern:

```
1 MATCH
2 (a)-->(b)-->(c)<--(a)
3 RETURN a.name, b.name, c.name;
```

Return nodes and lengths for paths between nodes 1 and 7 with max length of 4:

```
1 MATCH
2 p=(a)-[*1..4]-(b)
3 WHERE
4 a.name = 1 AND b.name = 7
5 RETURN nodes(p), length(p);
```

Return shortest Path between 1 and 7:

```
1 MATCH
2 p=shortestPath((a)-[*..4]-(b))
3 WHERE
4 a.name = 1 AND b.name = 7
5 RETURN nodes(p);
```

MongoDB

The Database is in the format:

```
1 {"city":"Tokyo",
2   "population":13960000,
3   "continent":"Asia",
4   "country":"Japan",
5   "known_for":["technology",
6               "finance",
7               "anime culture"],
8   "coordinates":{"lat":35.6762,
9                 "long":139.6503},
10  "timezone":"Asia/Tokyo"
11 }
```

Note: the city data was generated with ChatGPT, and I haven't checked the data. Hallucination levels may range from 1 beer to 5 tabs of ACID. But ACID is good for databases, so I hope it's OK.

Queries Get all cities:

```
1 db.cities.find()
```

Find in Europe (only print city name):

```
1 db.cities.find({ continent : 'Europe' }, { _id:0, city : 1})
```

Return cities known for history or government:

```
1 db.cities.find({known_for: {$in : ['history', 'government']}}, { _id :  
  0, known_for : 1 })
```